AI Framework Comparison: LangChain vs LangGraph vs LlamaIndex vs Haystack vs AutoGen

Table 1: Core Architecture & Design Philosophy

Feature	LangChain	LangGraph	LlamaIndex	Haystack	AutoGen
Primary Use Case	General LLM app development, chains & agents	Stateful multi-actor agent workflows with graph-based control flow	Data ingestion, indexing, and RAG pipelines	Production NLP pipelines and search systems	Multi-agent conversation frameworks
Core Abstraction	Chain / Runnable interface (LCEL)	StateGraph with nodes and edges	Index / QueryEngine / RetrieverQueryEngine	Pipeline with Components	ConversableAgent with async messaging
State Management	Minimal — stateless by default, memory via add-ons	First-class — typed state dict passed between nodes	Index-level state, no native agent state	Pipeline-level, component outputs passed forward	Conversation history per agent
Graph / DAG Support	No native graph; linear chains or agent loops	Full directed graph with conditional edges and cycles	No — pipeline is tree-structured	DAG-based pipeline, no cycles	Implicit conversation graph via message passing
Async Support	Yes — ainvoke, astream	Yes — full async graph execution	Yes — async query and retrieval	Yes — async component execution	Yes — built on asyncio natively
Streaming	Token streaming via astream_events	Node-level and token-level streaming via astream	Token streaming in query responses	Limited streaming support	Message streaming between agents
Open Source	Yes — MIT License	Yes — MIT License	Yes — MIT License	Yes — Apache 2.0	Yes — MIT License
Primary Language	Python (JS port available)	Python (JS port available)	Python (TS port available)	Python only	Python only
Backed By	LangChain Inc.	LangChain Inc.	LlamaIndex Inc.	deepset	Microsoft Research

Table 2: RAG & Retrieval Capabilities

RAG Feature	LangChain	LangGraph	LlamaIndex	Haystack	AutoGen
Vector Store Integrations	50+ (Pinecone, Chroma, FAISS, Weaviate, MongoDB Atlas, pgvector, Qdrant...)	Via LangChain integrations — inherits full ecosystem	35+ (Pinecone, Weaviate, Qdrant, MongoDB, pgvector...)	20+ (Elasticsearch, OpenSearch, Weaviate, Qdrant...)	Via custom tools — no native vector store
Chunking Strategies	RecursiveCharacterTextSplitter, TokenTextSplitter, SemanticChunker, MarkdownSplitter	Uses LangChain splitters	SentenceSplitter, TokenTextSplitter, SemanticSplitterNodeParser, HierarchicalNodeParser	DocumentSplitter with word/sentence/passage granularity	No native chunking — relies on external tools
Hybrid Search	Yes — with EnsembleRetriever (BM25 + dense)	Via LangChain retrievers	Yes — native hybrid with QueryFusionRetriever	Yes — native BM25 + dense hybrid	No native support
Re-ranking	Via Cohere, FlashRank retrievers	Via LangChain	Native — CohereRerank, SentenceTransformerRerank, LLMRerank	Yes — built-in reranker components	No native support
Multi-document RAG	Yes — MultiQueryRetriever, ParentDocumentRetriever	Agentic multi-doc via graph nodes	Yes — SubQuestionQueryEngine, RouterQueryEngine	Yes — pipeline supports multiple document stores	Via agent tool calling
Metadata Filtering	Yes — via retriever filter kwargs	Via LangChain retrievers	Yes — rich metadata filter API	Yes — filter expressions in pipeline	No native support
Document Loaders	100+ loaders (PDF, DOCX, CSV, HTML, Notion, GDrive...)	Via LangChain loaders	50+ loaders with SimpleDirectoryReader	20+ converters (PDF, DOCX, HTML, CSV...)	No native loaders

Evaluation Tools	LangSmith eval suite	LangSmith + graph trace	Native RAGAs-compatible eval, Faithfulness/Relevancy metrics	BEIR benchmark support, custom eval pipeline	AgentEval framework

Table 3: Agent Architecture & Tool Integration

Agent Feature	LangChain	LangGraph	LlamaIndex	Haystack	AutoGen
Agent Types	ReAct, OpenAI Functions, XML, Structured Chat, Self-Ask	Custom graph-based agents, create_react_agent, create_tool_calling_agent	ReActAgent, OpenAIAgent, FunctionCallingAgent	No dedicated agent — pipeline-based	AssistantAgent, UserProxyAgent, GroupChat
Multi-Agent Support	Limited — via custom chains	Native — multiple nodes as agents, supervisor pattern	Limited — AgentWorkflow with handoffs	No native multi-agent	Core feature — GroupChatManager, nested agents
Tool Definition	@tool decorator, StructuredTool, BaseTool	LangChain tools + ToolNode	FunctionTool, QueryEngineTool, @tool decorator	ComponentTool, custom tool classes	register_for_llm / register_for_execution decorators
Human-in-the-Loop	Via interrupt callbacks	Native — interrupt() primitive, breakpoints	Via HumanInputQueryEngine	No native support	UserProxyAgent handles human input natively
Memory Types	ConversationBufferMemory, SummaryMemory, VectorStoreMemory, EntityMemory	Checkpointer-based (InMemory, SQLite, MongoDB, PostgreSQL)	ChatMemoryBuffer, VectorMemory, SimpleComposableMemory	No dedicated memory — state via pipeline	Conversation history per agent, teachable agent
Persistence / Checkpointing	No native — requires external DB	Native checkpointer — resume from any node	No native checkpointing	No native checkpointing	No native checkpointing
Observability	LangSmith — full trace, token count, latency	LangSmith — graph-level trace with node visibility	LlamaTrace (Arize Phoenix), Langfuse integration	Haystack YAML pipeline export, custom callbacks	AutoGen Studio UI, logging callbacks
Code Execution	Via PythonREPLTool	Via LangChain tools	Via CodeInterpreterTool	No native support	Native — UserProxyAgent executes code in sandbox